BDA MINI PROJECT REPORT

City Problems

Mumbai Civic Issues Map

Submitted in partial fulfilment of the requirements
for the Semester VII BDA Mini Project
Bachelor of Engineering in Computer Engineering

Submitted by

Ayan Parkar (63)

Sourabh Patel (64)

Mohammed Owaiz Shaikh (65)

Under the guidance of

Prof. Puja Sarage



Department of Computer Engineering

Shree L. R. Tiwari College of Engineering

Kanakia Park, Mira Road (E), Thane - 401 107, Maharashtra

Academic Year 2026-2027

BDA Mini Project Report Approval

This project report entitled **“City Problems - Mumbai Civic Issues Map”**, submitted by **Mr. Ayan Parkar (63), Mr. Sourabh Patel (64)** and **Mr. Mohammed Owaiz Shaikh (65)**, is approved towards fulfilment of the Semester VII BDA Mini Project requirements for the Bachelor of Engineering in Computer Engineering at Shree L. R. Tiwari College of Engineering for the academic year 2026-2027.

_____

Internal Examiner

Declaration

We declare that this submission represents our original work. Wherever the ideas, words, data or other material of others have been used, the original sources have been appropriately cited and acknowledged.

We have adhered to the principles of academic honesty and integrity and have not knowingly misrepresented, fabricated or falsified any idea, data, fact or source in this report.

We understand that any violation of these principles may result in disciplinary action by the Institute and, where applicable, action by the original rights holders for material used without appropriate acknowledgement or permission.

_____

Signature - Mr. Ayan Parkar (63)

_____

Signature - Mr. Sourabh Patel (64)

_____

Signature - Mr. Mohammed Owaiz Shaikh (65)

Date: _____
Place: _____

Table of Contents

SECTION	PAGE
1. Introduction	5
2. System Architecture	7
3. Implementation	11
Technologies Used	15
4. Results	16
5. Conclusion	28
6. References	29

1. Introduction

Mumbai's civic body, the Brihanmumbai Municipal Corporation (BMC), receives tens of thousands of citizen complaints every few months: potholes, uncollected garbage, blocked drains, water-supply faults, broken street lights, encroachment and more. Each complaint is a small record, but together they form a large, geo-tagged, time-stamped dataset that shows where the city is failing, how fast problems are fixed, and how complaint patterns change with seasons such as the monsoon.

City Problems is a Big Data Analysis (BDA) mini-project that turns this public data into a map-based application for the people who pay for these services: Mumbai's residents and taxpayers. A resident picks an area by searching, using their location, tapping the map or choosing a ward. The app then shows open problems nearby, how quickly BMC fixes problems in that ward compared with the rest of the city, and what neighbours have reported. Residents can report a new problem by placing a pin on the map. An analyst view, the Data Explorer, keeps the group-by and aggregation workbench of the first version and shows the equivalent PostgreSQL query for every result.

1.1 Big data perspective

- **Volume:** 124,146 raw records were downloaded (112,086 general complaints and 12,060 pothole records). After joining and cleaning they form one table of 112,094 complaints, of which 96,998 have a real map location.
- **Variety:** Structured complaint rows (category, ward, status, timestamps, filing channel), geospatial points, ward polygons with population, and free-text reports written by residents.
- **Velocity:** The BMC MARG service keeps receiving complaints and changing statuses, and residents add reports in real time through the app. The snapshot used here was fetched on 11 Oct 2026; the pipeline can be re-run to refresh it.
- **Veracity:** The raw data has placeholder coordinates, complaints closed as invalid, UTC timestamps and inconsistent department routing. The cleaning pipeline (Section 2.2) corrects each of these before analysis.

1.2 Objectives

- Acquire a real, geo-tagged Mumbai civic-complaint dataset from a public source.
- Clean and integrate multiple layers into one uniform dataset and document the decisions with exploratory data analysis.
- Give residents a meaningful view of their own area: open problems, fix rate and time to fix, compared with the city.

- Let residents report a problem by selecting its location on the map, and store reports in a PostgreSQL database (Neon).
- Keep an analyst workbench with flexible group-by queries, eight preset questions and the equivalent SQL.

2. System Architecture

2.1 Data source

The data comes from the public MARG ArcGIS REST service of BMC [1], the same service used by BMC's own Pothole QuickFix dashboard [2]. Three layers are used:

- **Layer 0 COMPLAINTS_REGISTRATION:** 112,086 complaints from every department and every filing channel, each with its own latitude and longitude.
- **Layer 16 POTHOLE_REGISTRATION:** 12,060 pothole records from Pothole QuickFix, with road type and an invalid flag.
- **Layer 2 Ward_Boundary:** 26 ward polygons with a population field.

Licence note: BMC states no explicit open-data licence on this service. It is public, unauthenticated government data and is used here only for non-commercial academic purposes with attribution to BMC/MCGM. Only non-personal fields were downloaded (no phone numbers, free-text descriptions, officer or contractor names, or photos). Other candidate datasets were reviewed and not used: the Kaggle "1.2M Complaints BMC" set is documented as synthetic, Praja Foundation's 2026 civic report [3] publishes city-level totals in PDF form rather than individual complaints, and OpenCity's grievance data covers Bengaluru.

2.2 Data pipeline (scripts/download_raw.py, scripts/build_dataset.py, scripts/eda.py)

The pipeline is written in Python with pandas and Shapely; the EDA charts use Matplotlib. Figure 1 shows the pipeline. The main decisions, each backed by the exploratory analysis in data/eda/EDA.md, are:

1. One register, not two: 12,037 of the 12,060 pothole records carry the same complaint ID as a layer 0 complaint, so the layers are two views of one register. They are joined on ID and the pothole fields are copied onto the layer 0 row. 9 pothole-only records are added and 10 without an ID are dropped. The first version stacked both layers and de-duplicated, which silently discarded the pothole fields.

2. Closed is not the same as fixed: BMC marks 24,626 complaints as CLOSED while also flagging them INVALID. The first version counted them as resolved. The unified status separates Resolved (52,261) from Rejected (24,626), and the fix rate is Resolved ÷ (Resolved + Rejected).

3. Indian time: MARG stores UTC epoch milliseconds. All timestamps are converted to IST, which moves late-evening complaints to the correct day; the busiest hour becomes 11:00 IST instead of 05:00 UTC.

4. Time to fix: days_to_close runs from the report to the date of BMC's "after" photo (the resolved date for potholes). Values below 0 or above 400 days are blanked.

5. Locations: 14,546 complaints, mostly from the 1916 helpline, sit on one placeholder point (19.07766, 72.87741) and 550 lie outside Greater Mumbai. They stay in the table with has_location = false so city totals remain complete, but they are not drawn on the map.

6. Wards: 58 complaints with a missing ward are assigned one by point-in-polygon on the BMC ward boundaries (Shapely STRtree).

7. Citizen-facing categories: Categories come from the complaint name the citizen picked, with BMC's department as a fallback, because the same complaint name is often routed to different departments. The result is 14 categories such as Potholes, Drains & Sewage and Illegal Construction.

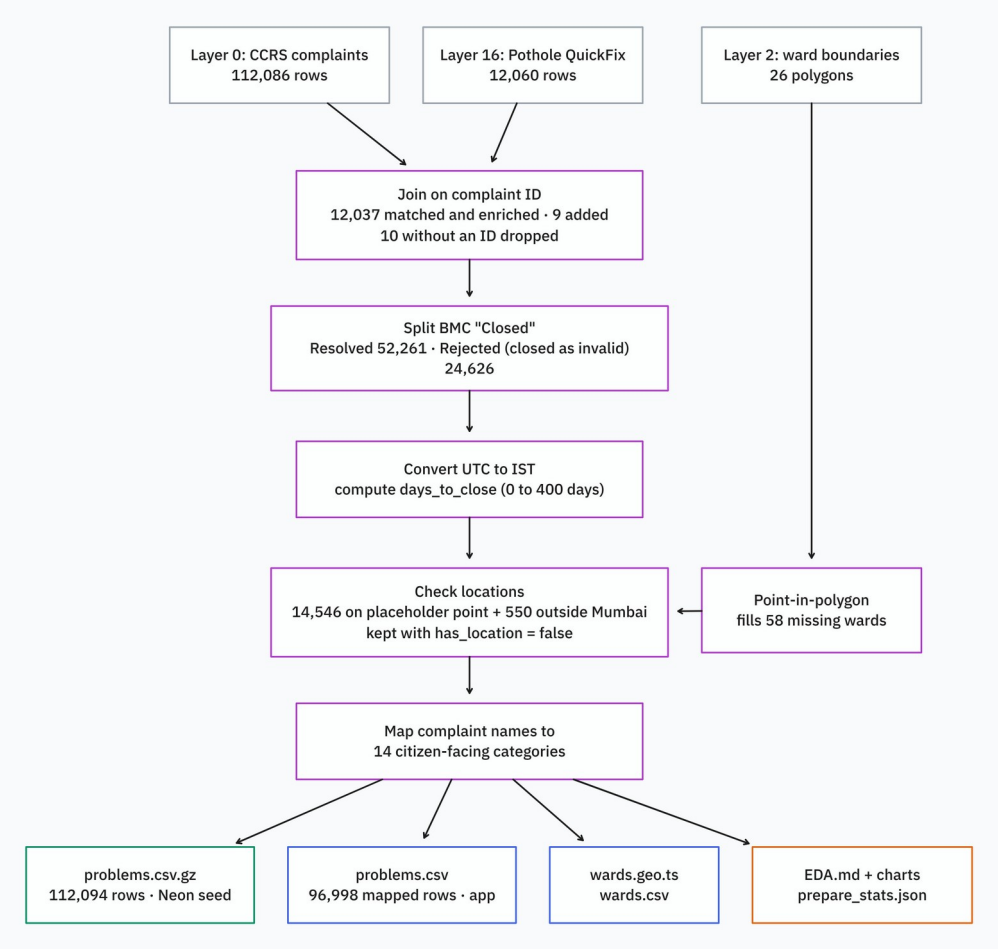


Fig. 1: Data pipeline from the three MARG layers to the unified dataset and its outputs

Outputs: data/processed/problems.csv.gz (112,094 rows, 20 columns, the database seed), web/public/data/problems.csv (the 96,998 mapped rows in compact form, loaded by the browser), the simplified ward outlines in web/shared/wards.geo.ts, data/processed/wards.csv and data/prepare_stats.json with the row counts for every step.

2.3 Application architecture

The web application is a React single-page app built with Vite and deployed on Vercel with web/ as the project root (Figure 2). BMC complaints are served as a static CSV from the CDN and analysed in the browser, which keeps the map fast without a database round trip. Resident reports go through Vercel functions in web/api/ to a Neon PostgreSQL database, which also holds the full unified dataset for SQL analysis. During local development the same functions run inside the Vite dev server against an embedded PGlite database, and without a database the app saves reports on the visitor's device and says so.

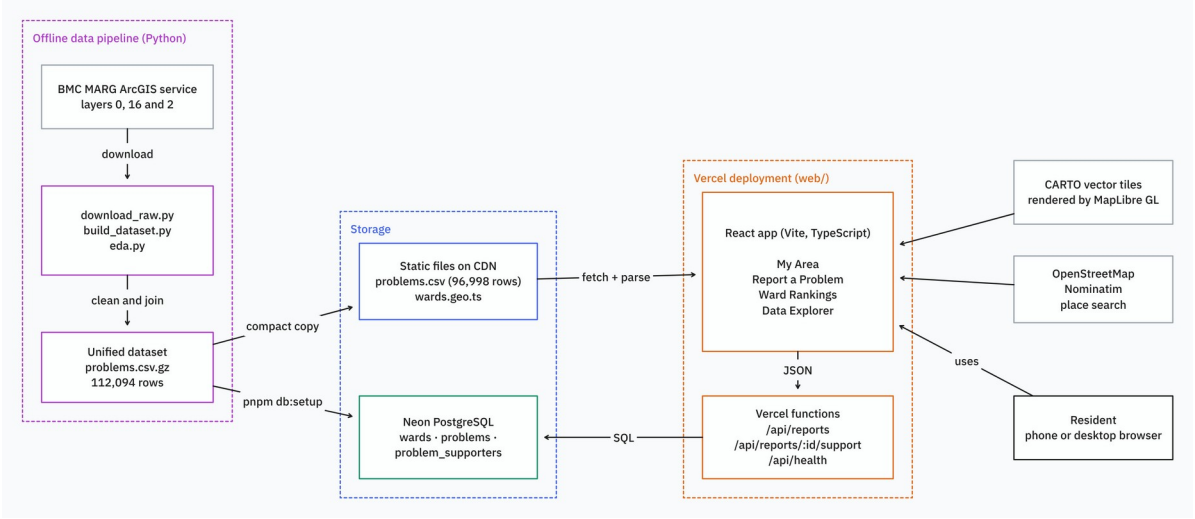


Fig. 2: System architecture: offline pipeline, storage, Vercel deployment and external map services

2.4 Database design

One problems table holds both BMC complaints (source = 'bmc') and resident reports (source = 'citizen'), so every query and ranking can include either. The wards table holds names, well-known areas and population for per-capita rates. problem_supporters records one "I face this too" vote per device. is_open is a generated column, and a check constraint ties has_location to the presence of coordinates (Figure 3).



Fig. 3: Database schema in Neon PostgreSQL (web/db/schema.sql)

2.5 Main modules

- **views/AreaView.tsx:** My Area: place search through Nominatim, location, ward picker, ward statistics with rank and city comparison, and the list of problems within 250 m, 500 m or 1 km.

- **views/ReportView.tsx:** Report a Problem: centre-pin placement, category, description and landmark, validation messages and the tracking ID.

- **views/OverviewView.tsx:** Ward Rankings: all wards ranked and mapped by fix rate, median days to fix, open complaints or complaints per 100,000 residents.

- **views/ExplorerView.tsx, query.ts:** Data Explorer: filters, group by, split by, measure, Top-N, minimum group size, hotspot rule, chart, table, CSV export and the equivalent SQL.

- **components/MapView.tsx:** MapLibre GL map (mapcn component) with GPU clustering of complaints, resident-report markers, ward outlines and the choropleth.

- **api/reports, api/_lib:** Vercel functions: list, create and support resident reports; validation, rate limit and server-side ward assignment.

- **shared/:** Code used by both the app and the API: categories, ward outlines and point-in-polygon geometry.

3. Implementation

3.1 Categorisation in the cleaning pipeline

```
CATEGORY_RULES = [  # (category, regex on complaint name) - first match wins
    ("Potholes", r"pothole"),
    ("Street Lights", r"street light|electric supply"),
    ("Encroachment & Hawkers", r"hawk|encro|banner|advertisement"),
    ("Drains & Sewage", r"drain|manhole|nullah|nalla|sewer|sewage|septic|swd"),
    ("Water Supply", r"water (supply|line|connection|bill|meter)|contaminated water"),
    ("Garbage & Cleanliness", r"garbage|debris|swept|sweeping|dustbin|silt"),
    ...
]
def categorise(dept: str, name: str) -> str:
    n = name.lower()
    if dept in PUBLIC_DEPTS or n.startswith(("school -", "health -")):
        return "Public Buildings & Services"
    for cat, rx in CATEGORY_RULES:
        if re.search(rx, n):
            return cat
    return DEPT_FALLBACK.get(dept, "Other")
```

3.2 Joining the layers and splitting the status

```
overlap = p.id.isin(c.id)               # 12,037 pothole IDs are already in layer 0
c = c.merge(p[overlap], on="id", how="left")
m = c.p_status.notna()
c.loc[m, "road_type"] = c.loc[m, "p_road_type"]
c.loc[m & c.p_invalid, "invalid"] = True # QuickFix flags potholes CCRS still calls Closed

def status(row):
    s = row.bmc_status
    if s == "CLOSED": return "Rejected" if row.invalid else "Resolved"
    return {"IN PROGRESS": "In Progress", "IN REVIEW": "In Review",
            "NEW": "Open", "DISCARDED": "Discarded"}.get(s)

df["reported_at"] = (pd.to_datetime(ms, unit="ms", utc=True)
                     .dt.tz_convert("Asia/Kolkata"))
df["has_location"] = inside_mumbai & ~on_placeholder_point
```

3.3 Loading the data in the browser

The browser fetches the compact CSV once and parses it with Papa Parse.

```
export async function loadProblems(): Promise<Problem[]> {
  const res = await fetch(`${import.meta.env.BASE_URL}data/problems.csv`);
  const parsed = Papa.parse<Record<string, string>>(await res.text(),
                                                    { header: true, skipEmptyLines: true });
  return parsed.data.map((r) => ({
    id: r.id, date: r.d, month: r.d.slice(0, 7), hour: +r.h,
    category: r.c, subcategory: r.s, ward: r.w, status: r.st,
    isOpen: OPEN_STATUSES.has(r.st), lat: +r.lat, lon: +r.lon,
    daysToClose: r.dc === "" ? null : +r.dc, channel: r.ch, escalated: r.esc === "1",
  }));
}
```

3.4 Reporting a problem

Figure 4 shows the flow. The browser validates the form first; the Vercel function validates again, assigns the ward on the server, limits each device to 10 reports a day and inserts the row with a human-readable tracking ID. Reporters are identified only by a salted hash of a random device ID and the IP address; no name, phone number or raw IP is stored. All user text is rendered as text, never as HTML.

```
export function validateReport(body: Record<string, unknown>): NewReport | string {
  const category = clean(body.category, 60);
  if (!REPORTABLE.has(category)) return "Choose a problem type from the list.";
  const description = clean(body.description, LIMITS.description);
  if (description.length < 10) return "Describe the problem in at least 10 characters.";
  const lat = Number(body.lat), lon = Number(body.lon);
  if (!insideMumbai(lat, lon))
    return "The pin is outside Mumbai. Move the pin to the problem location.";
  const ward = wardAt(wards.features, lat, lon);
  if (!ward)
    return "The pin is outside BMC's 26 wards, for example in the sea. Move the pin onto
        land.";
  const landmark = clean(body.landmark, LIMITS.landmark);
  return { category, description, landmark, lat, lon, ward };
}
```

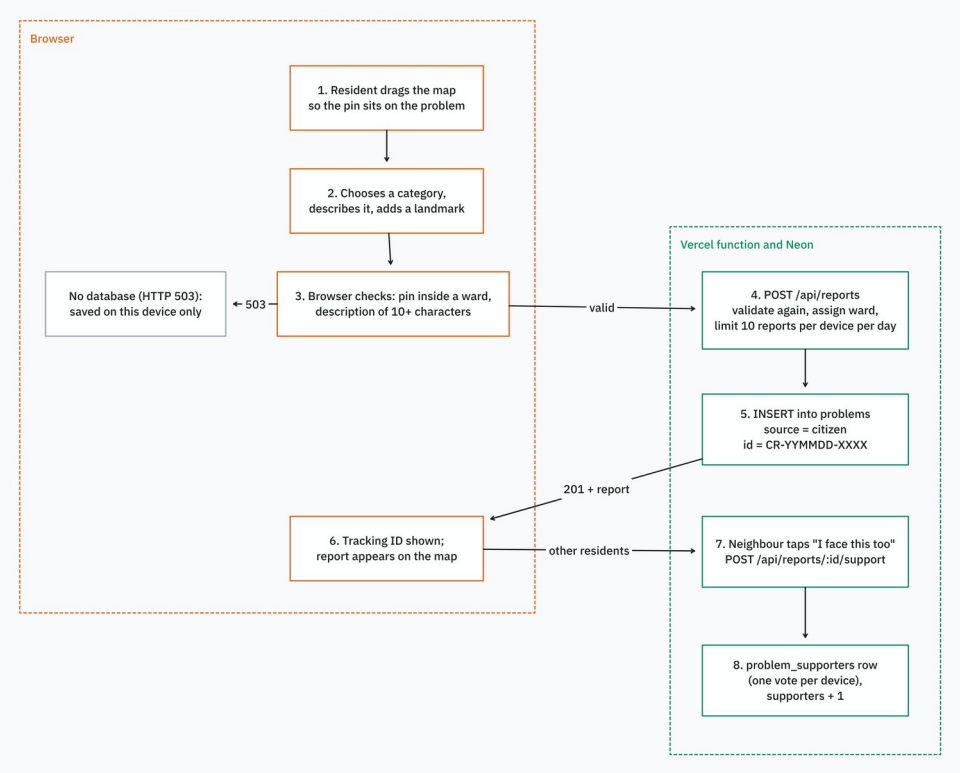


Fig. 4: Reporting a problem and adding “I face this too”

3.5 Ward lookup with a border tolerance

Ward outlines are simplified for the browser, which leaves slivers of a few metres between neighbouring wards. A point that falls in such a gap, for example at Andheri station, is assigned to the nearest ward within 150 m. On a sample of 4,850 complaints this matches BMC’s own ward label 98% of the time.

```
export function wardAt(features: WardFeature[], lat: number, lon: number,
                       toleranceM = 150): string | null {
  for (const f of features)
    if (polygonsOf(f.geometry).some((p) => inPolygon(lon, lat, p))) return
        f.properties.ward;
  let best: { ward: string; d: number } | null = null;
  for (const f of features)
    for (const p of polygonsOf(f.geometry)) {
      const d = ringDistanceM(lon, lat, p[0]);
      if (d <= toleranceM && (!best || d < best.d)) best = { ward: f.properties.ward, d };
    }
  return best?.ward ?? null;
}
```

3.6 Group-by aggregation

```
function value(a: Acc, m: Metric, pop?: number): number | null {
  switch (m) {
    case "count": return a.n;
    case "open_now": return a.open;
    case "fix_rate": {
      const closed = a.resolved + a.rejected;
      return closed ? +((100 * a.resolved) / closed).toFixed(1) : null;
    }
    case "median_days": { const v = median(a.days); return v === null ? null :
        +v.toFixed(1); }
    case "per_100k": return pop ? +((a.n * 100000) / pop).toFixed(1) : null;
  }
}
```

runQuery() makes one pass over the filtered rows, accumulating counts, open and closed counts and days to fix per group (and per split group), then applies the minimum group size (HAVING), the hotspot rule (value ≥ mean + 0.5σ), ordering and Top-N. Medians are used instead of averages because days to fix is heavily skewed.

3.7 Preset analytical queries

The Data Explorer offers eight preset questions. Each query below is the PostgreSQL the app generates for the Neon problems table; the same query runs in the browser over the static file. Results are in Section 4.

1. Wards With Most Potholes

```
SELECT ward AS ward,
       count(*) AS count
FROM   problems p
WHERE  has_location
  AND  category IN ('Potholes')
GROUP BY ward
ORDER BY count DESC NULLS LAST
LIMIT 10;
```

2. Potholes by Month

```
SELECT to_char(reported_at AT TIME ZONE 'Asia/Kolkata', 'YYYY-MM') AS month,
       count(*) AS count
FROM   problems p
WHERE  has_location
  AND  category IN ('Potholes')
GROUP BY to_char(reported_at AT TIME ZONE 'Asia/Kolkata', 'YYYY-MM')
ORDER BY 1;
```

3. Slowest Categories To Fix

```
SELECT category AS category,
       percentile_cont(0.5) WITHIN GROUP (ORDER BY days_to_close) FILTER (WHERE status =
           'Resolved') AS median_days,
       count(*) AS complaints
FROM   problems p
WHERE  has_location
GROUP BY category
HAVING count(*) >= 100
ORDER BY median_days DESC NULLS LAST;
```

4. Wards With Lowest Fix Rate

```
SELECT ward AS ward,
       round(100.0 * count(*) FILTER (WHERE status = 'Resolved') / nullif(count(*) FILTER
           (WHERE status IN ('Resolved', 'Rejected')), 0), 1) AS fix_rate,
       count(*) AS complaints
FROM   problems p
WHERE  has_location
GROUP BY ward
HAVING count(*) >= 200
ORDER BY fix_rate ASC NULLS LAST
LIMIT 10;
```

5. Hotspot Wards per 100k

```
SELECT ward AS ward,
       round(count(*) * 100000.0 / w.population, 1) AS per_100k,
       count(*) AS complaints
FROM   problems p JOIN wards w USING (ward)
WHERE  has_location
GROUP BY ward, w.population
ORDER BY per_100k DESC NULLS LAST;
-- then keep groups whose value ≥ mean + 0.5 × standard deviation
```

6. Open Backlog by Category

```
SELECT category AS category,
       count(*) FILTER (WHERE is_open) AS open_now,
       count(*) AS complaints
FROM   problems p
WHERE  has_location
GROUP BY category
ORDER BY open_now DESC NULLS LAST;
```

7. Complaints by Hour of Day

```
SELECT extract(hour FROM reported_at AT TIME ZONE 'Asia/Kolkata') AS hour,
       count(*) AS count
FROM   problems p
WHERE  has_location
GROUP BY extract(hour FROM reported_at AT TIME ZONE 'Asia/Kolkata')
ORDER BY 1;
```

8. Filing Channel per Category

```
SELECT category AS category,
       channel AS channel,
       count(*) AS count
FROM   problems p
WHERE  has_location
GROUP BY category, channel
ORDER BY count DESC NULLS LAST;
```

Technologies Used

React 19, Vite and TypeScript
Single-page application, typed data models and the production build.

Tailwind CSS v4 and shadcn/ui (Radix UI)
Styling and accessible interface components: buttons, selects, toggles, drawers, popovers and tables.

MapLibre GL with the mapcn component and CARTO basemaps
Vector map rendering, GPU clustering of 97,000 points, ward outlines and choropleths. Map data © OpenStreetMap contributors.

Recharts with evilcharts components
Bar, stacked bar and line charts in the Data Explorer.

PapaParse
CSV parsing in the browser.

OpenStreetMap Nominatim
Place search limited to Mumbai.

Vercel Functions and Neon PostgreSQL
API for resident reports and the database that stores the unified dataset and reports (@neondatabase/serverless driver).

PGlite
Embedded PostgreSQL used by the local development server and for testing the database seed.

Python 3, pandas, Shapely and Matplotlib
Data acquisition, cleaning, joining, point-in-polygon ward assignment and EDA charts.

BMC MARG ArcGIS REST service
Source of complaint records, pothole records and ward boundaries [1].

tldraw
Architecture, pipeline, schema and flow diagrams in this report.

pnpm and Playwright
Package management, and browser testing and screenshots of the app.

4. Results

4.1 Exploratory data analysis

The analysis covers all 112,094 complaints in the unified dataset. Figure 5 shows the volume by category. Encroachment & Hawkers and Garbage & Cleanliness lead with about 20,000 and 19,000 complaints, followed by Drains & Sewage and Potholes.

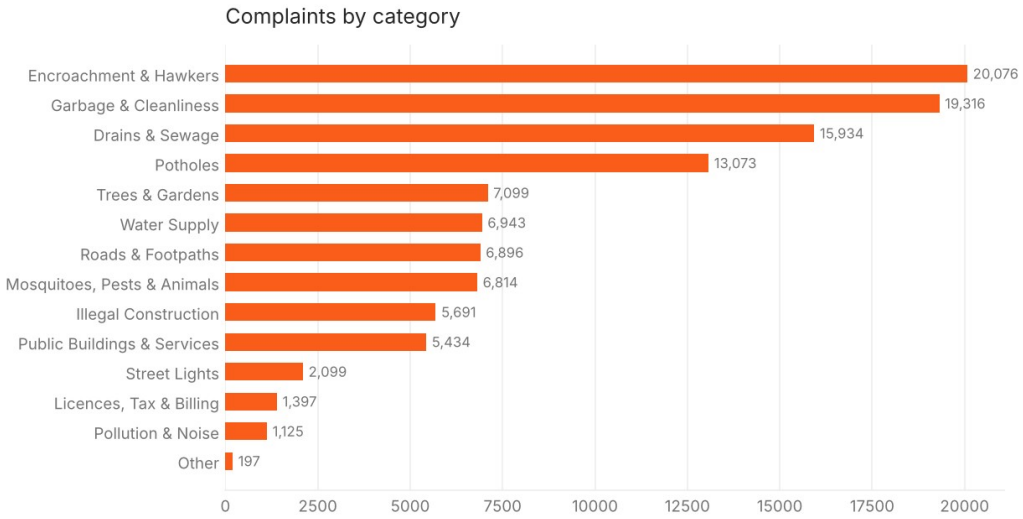


Fig. 5: Complaints by category (all 112,094 complaints)

Status	Complaints	Share
Resolved	52,261	46.6%
Rejected (closed as invalid)	24,626	22.0%
In Progress	14,326	12.8%
Discarded	13,875	12.4%
In Review	6,374	5.7%
Open	632	0.6%

Across the city BMC fixed 68.0% of the complaints it closed and rejected the other 32.0% as invalid. Potholes have the highest rejection share (39.7%), followed by Street Lights (37.8%). 21,332 complaints were still open on 11 Oct 2026.

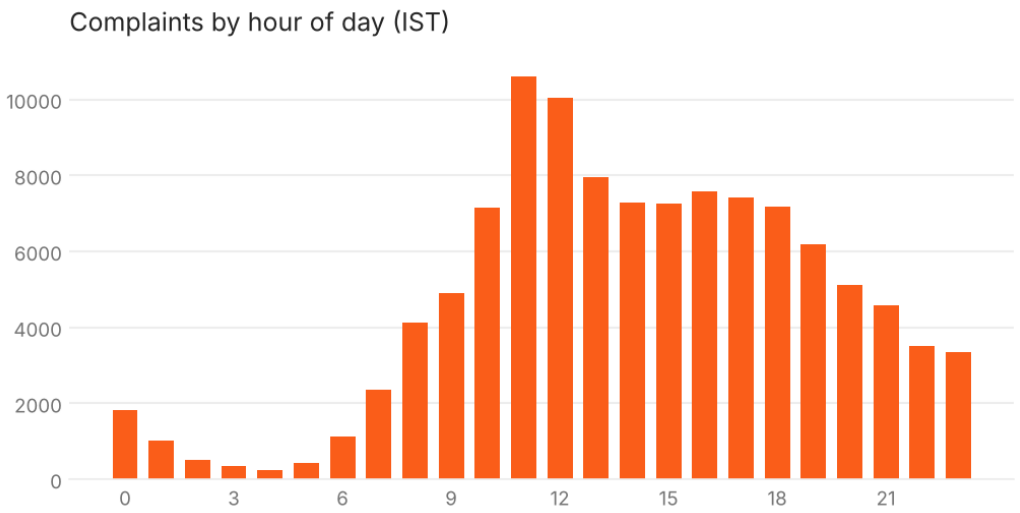


Fig. 6: Complaints by hour of day in IST; the peak is 11:00

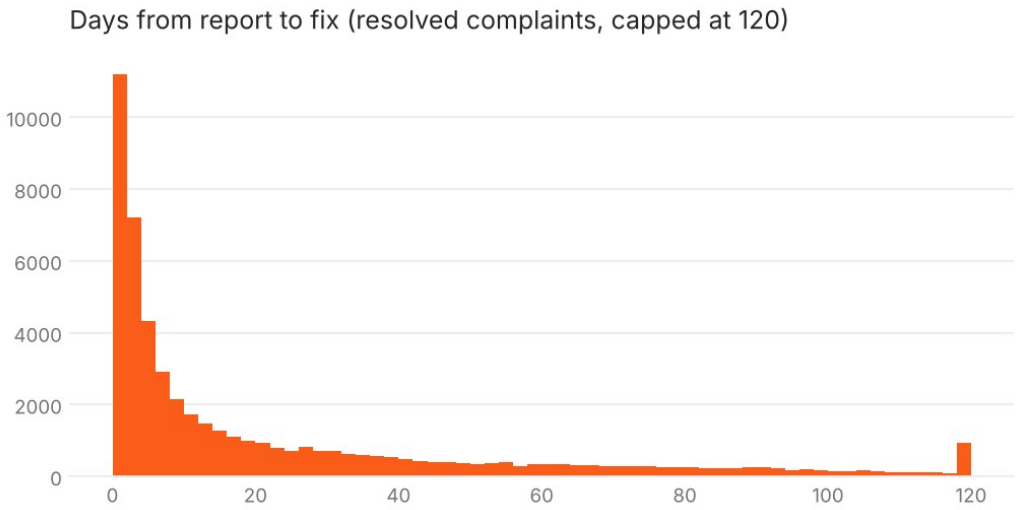


Fig. 7: Days from report to fix for resolved complaints (capped at 120 days)

Time to fix is heavily skewed: the median is 8.4 days but the mean is 23.3 days. By median, potholes are fixed fastest (2.0 days) and illegal construction slowest (45 days).

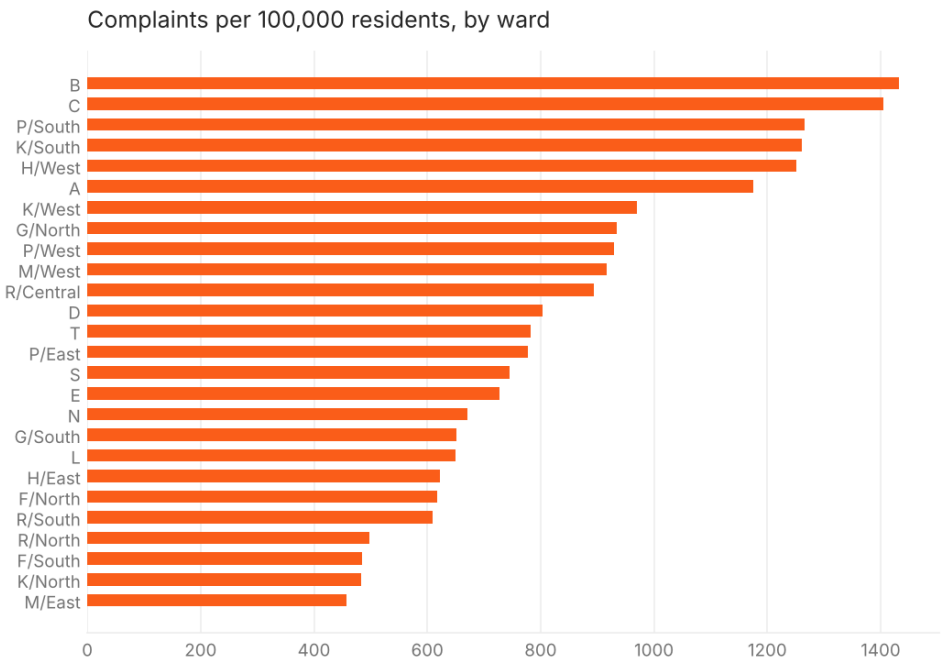


Fig. 8: Complaints per 100,000 residents by ward

Per-capita rates change the picture: the small island-city wards B and C file the most complaints per resident (about 1,400 per 100,000), while the large eastern suburb M/East files the fewest (458 per 100,000) despite its size.

4.2 The application

Screenshots of the running application follow. They were captured in a sandbox that could not reach the CARTO tile server, so the street basemap appears as plain land and sea colours; ward outlines, complaints and all interface elements are as deployed.

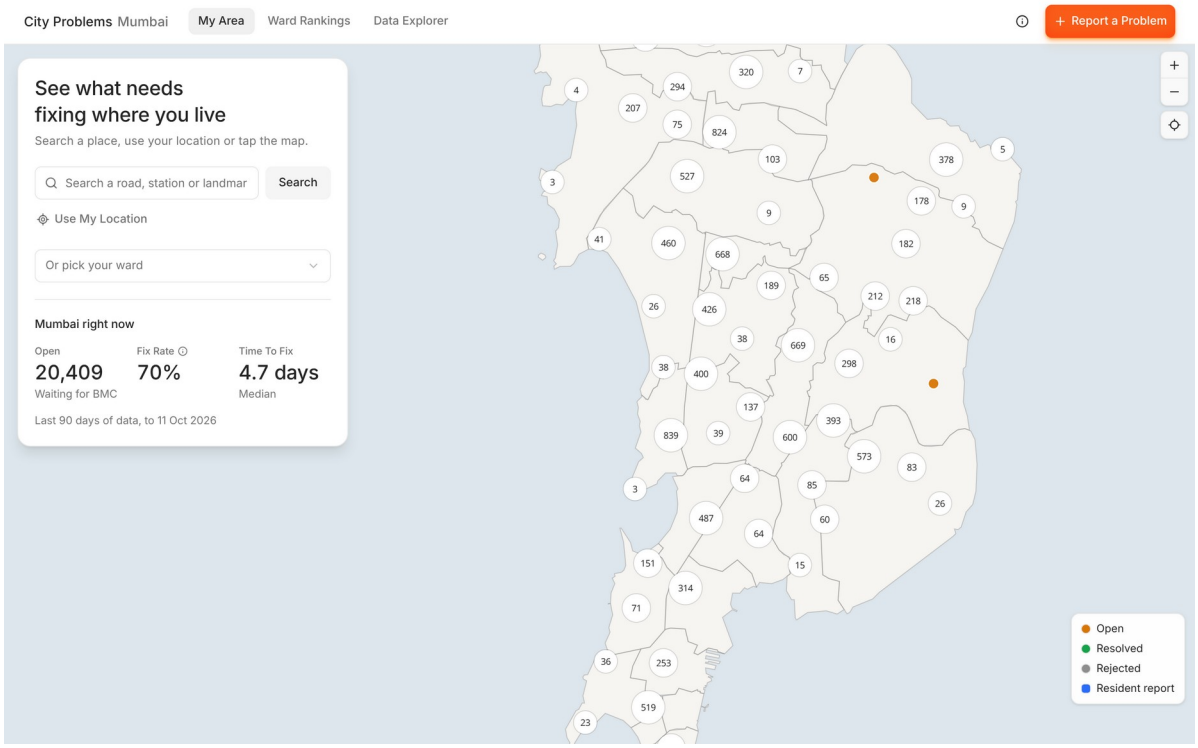


Fig. 9: My Area before an area is chosen: search, location, ward picker and city figures over a map of clustered complaints

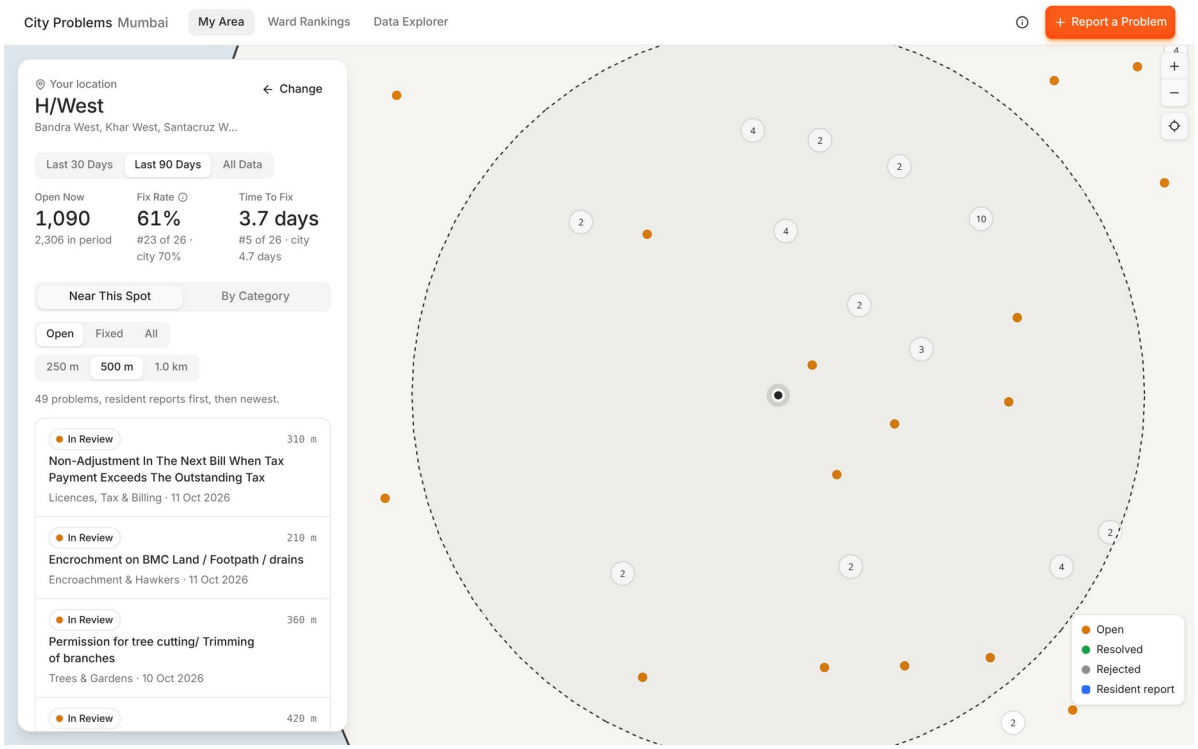


Fig. 10: My Area for H/West after "Use My Location": 1,090 open, fix rate 61% (rank 23 of 26), median 3.7 days to fix, and open problems within 500 m

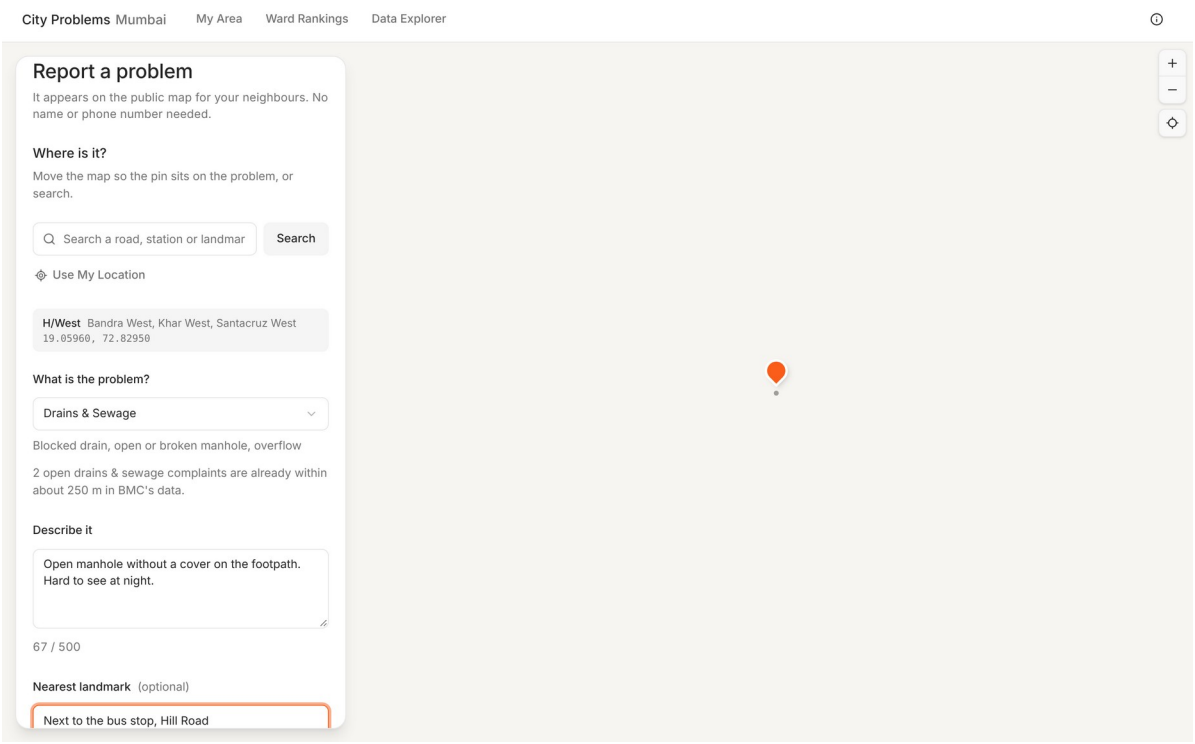


Fig. 11: Report a Problem: the resident moves the map under the pin; the form shows the ward and nearby open complaints of the same type

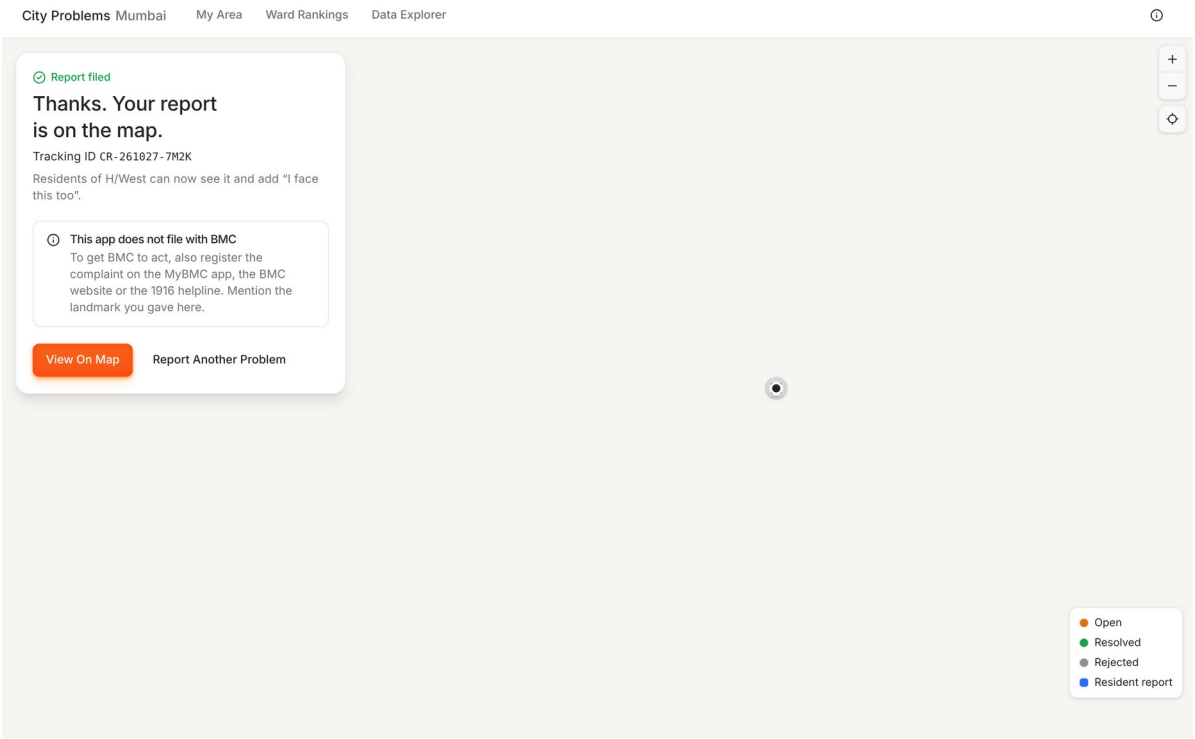


Fig. 12: Report confirmation with tracking ID and the reminder to also file with BMC

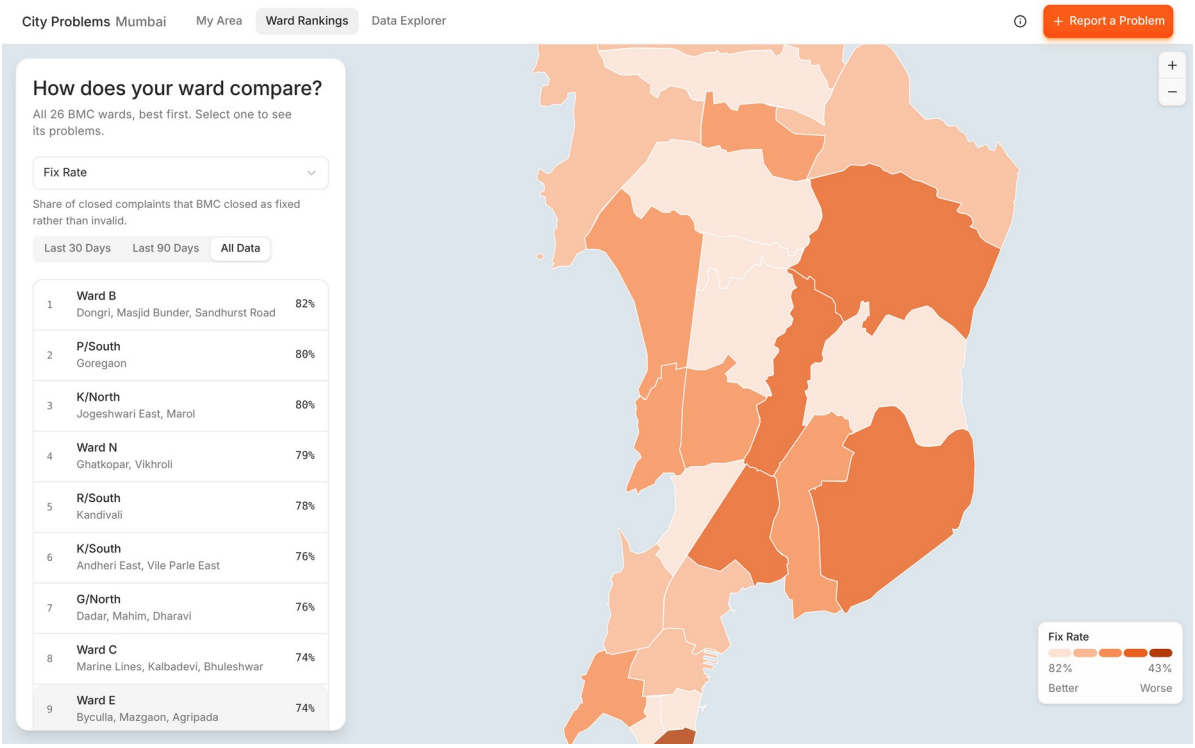


Fig. 13: Ward Rankings by fix rate: Ward B leads with 82%; Ward A is last with 43%

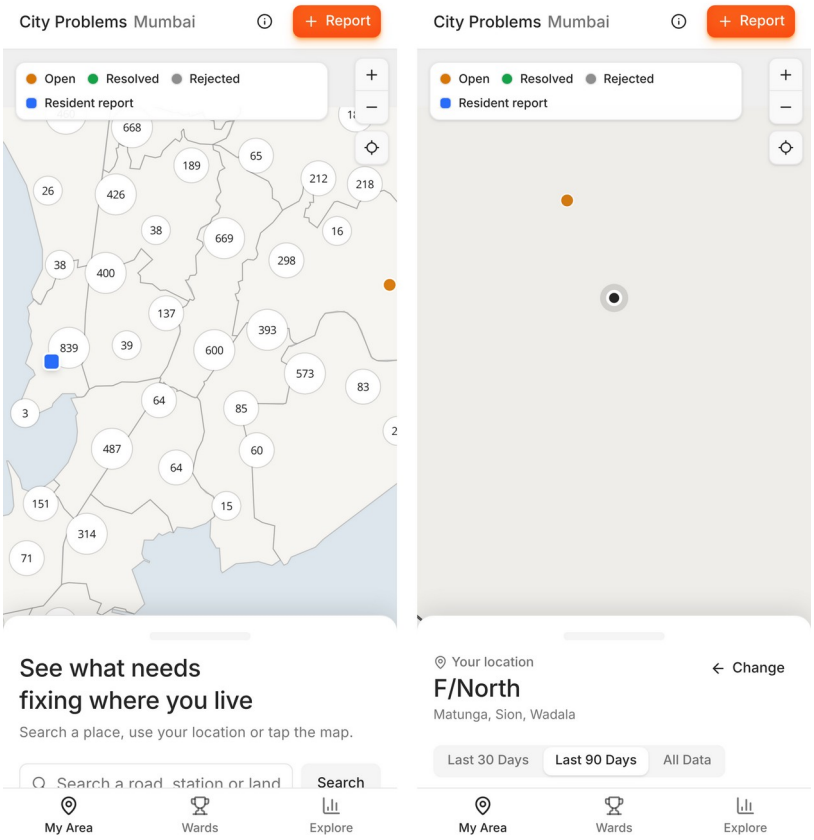


Fig. 14: Phone layout: map with a bottom sheet (left) and My Area for F/North (right)

4.3 Preset query results

The Data Explorer results below cover the 96,998 complaints with a real map location.

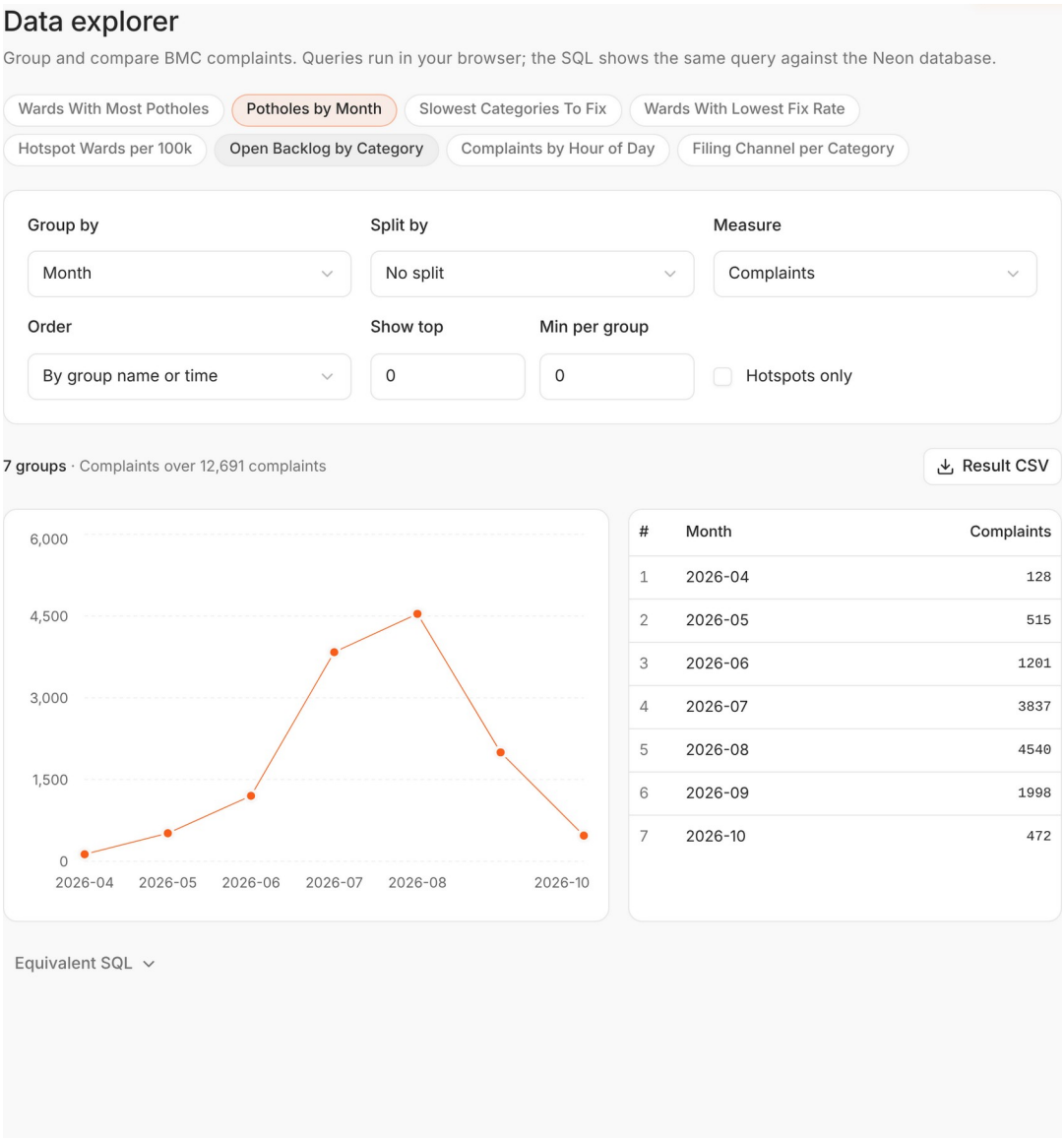


Fig. 15: Query 2, potholes by month: 128 in April, a peak of 4,540 in August, then 1,998 in September (October is partial)

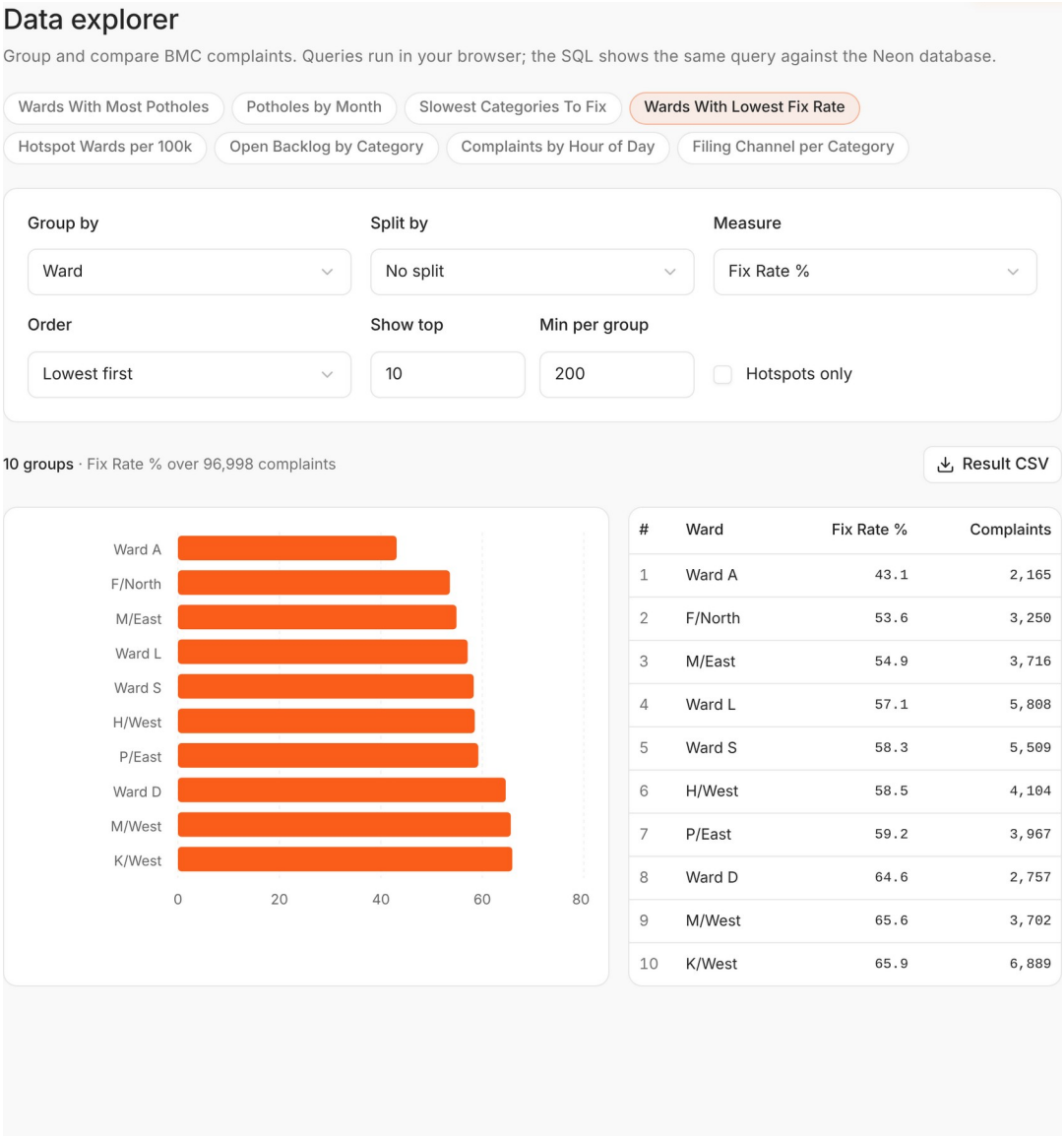


Fig. 16: Query 4, wards with the lowest fix rate: Ward A 43.1%, F/North 53.6%, M/East 54.9%

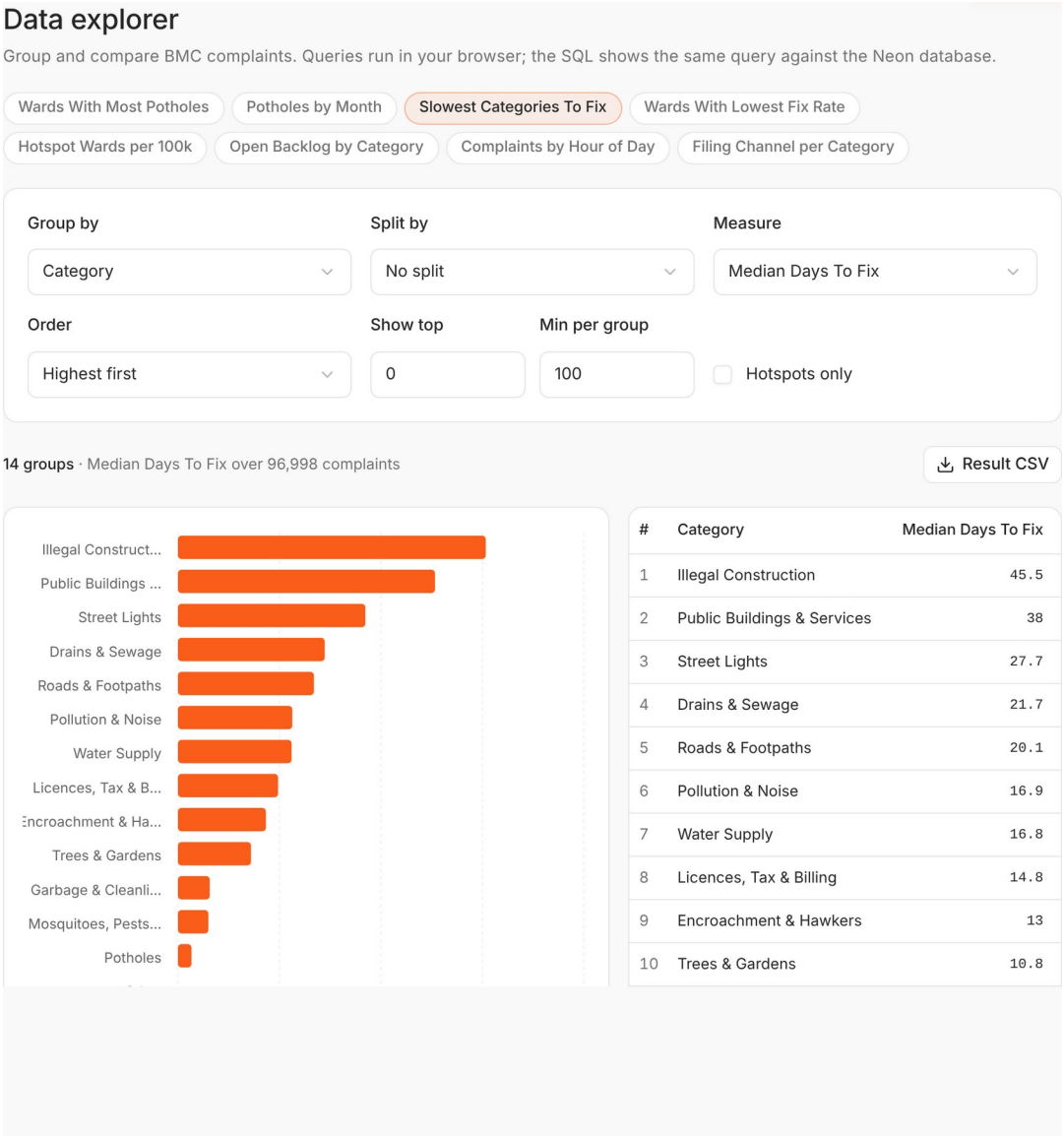


Fig. 17: Query 3, median days to fix by category: Illegal Construction 45.5 days; Mosquitoes, Pests & Animals 4.5 days

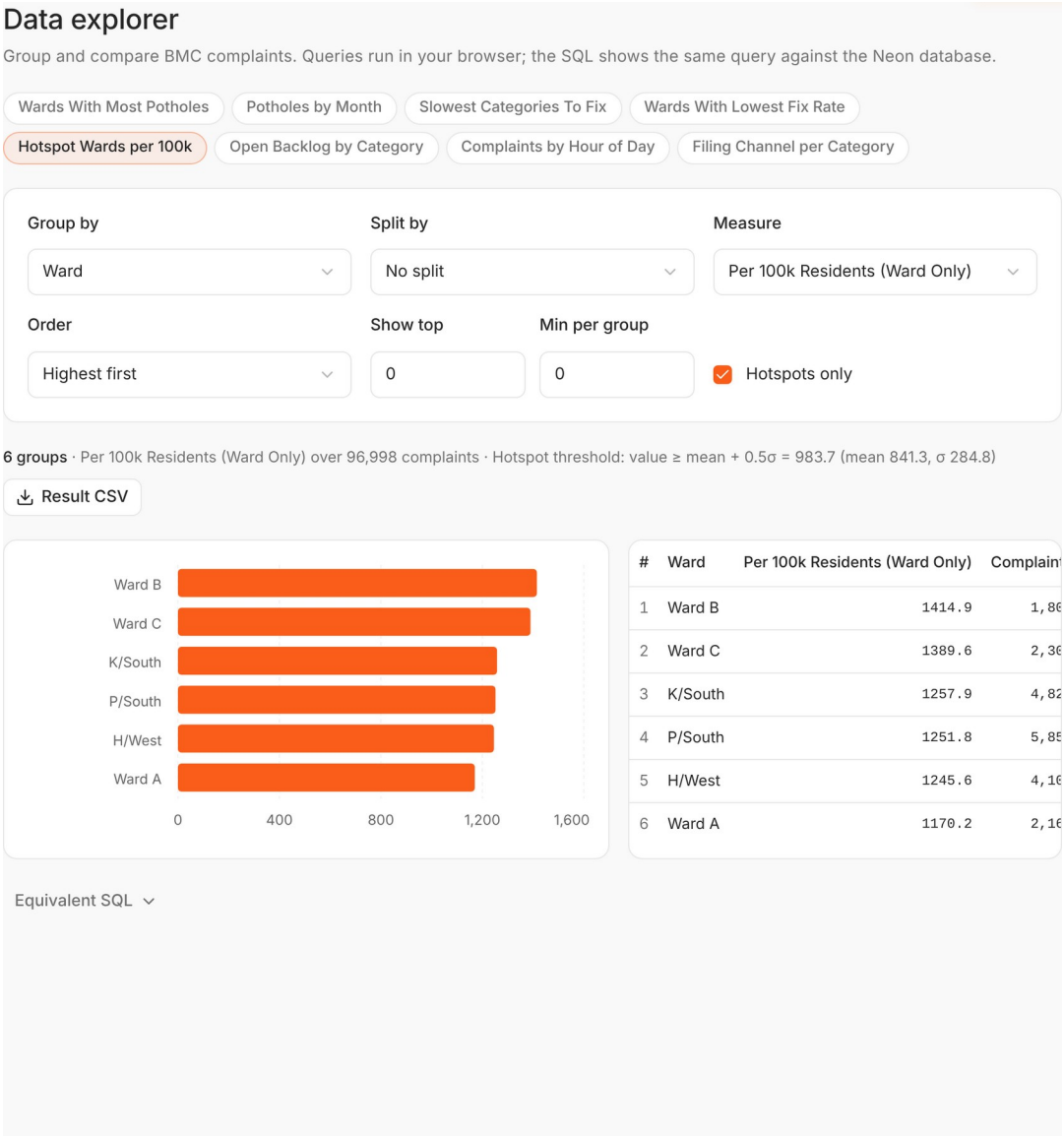


Fig. 18: Query 5, hotspot wards per 100,000 residents (threshold 983.7): B, C, K/South, P/South, H/West and A

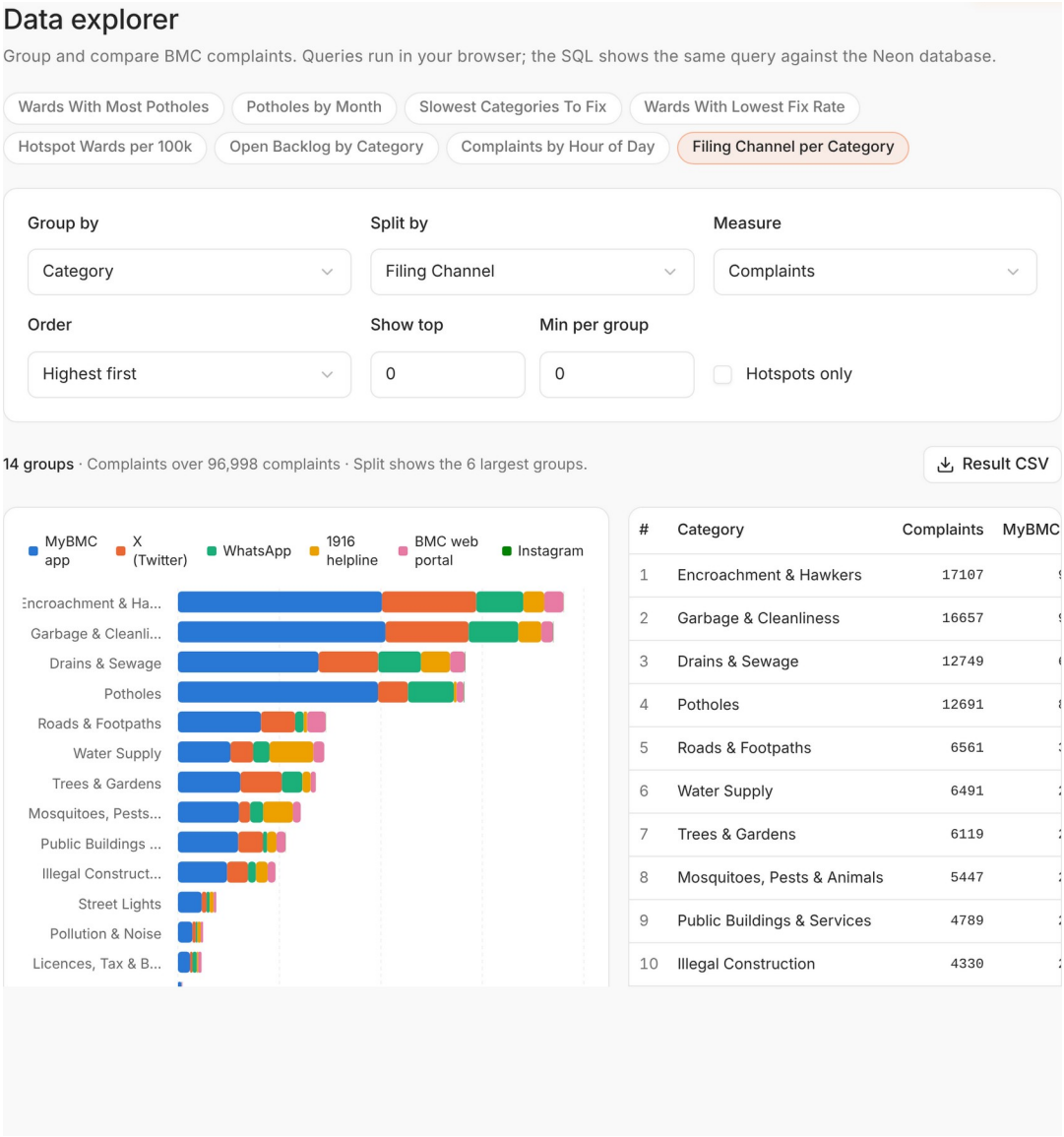


Fig. 19: Query 8, filing channel per category: the MyBMC app is the main channel for every category

Query	Main result
1. Wards with most potholes	Ward S 1,246; M/East 935; R/Central 891
6. Open backlog by category	Encroachment & Hawkers 4,242 open; Drains & Sewage 3,247; Roads & Footpaths 2,745
7. Complaints by hour (IST)	Peak at 11:00 (8,930); quietest at 04:00 (220)

Observations

• Closing a complaint is not the same as fixing it: almost a third of closed complaints were rejected as invalid, and this varies from 18% in Ward B to 57% in Ward A.

• Pothole complaints follow the monsoon, growing from April to a peak in August, and are fixed fastest (median 2 days) but also rejected most often.

• Illegal construction and repairs to public buildings take the longest to resolve (medians of 38 to 45 days).

• Per-capita rates identify the island-city wards B, C and A and the western suburbs K/South, P/South and H/West as hotspots.

• Residents file most complaints between 10:00 and 13:00 IST, mainly through the MyBMC app.

Limitations and caveats

• The data is a single seven-month snapshot (March to October 2026); October 2026 is a partial month.

• Complaints are not the same as problems: busier wards file more, and filing behaviour differs by channel.

• Time to fix uses the date of BMC's "after" photo because the service publishes no true closure date for general complaints.

• Rejected means BMC closed a complaint as invalid; the data does not say whether the citizen agreed.

• Reports filed in this app are not sent to BMC; the app tells residents to also use the MyBMC app, the BMC website or the 1916 helpline.

• No explicit BMC licence: the data is used for non-commercial academic purposes with attribution.

5. Conclusion

City Problems demonstrates an end-to-end civic-data workflow on the BMC MARG ArcGIS service: acquisition of 124,146 raw records, integration into one uniform table of 112,094 complaints, exploratory analysis, and an application that residents can use.

The second version moves the focus from analysts to residents. A taxpayer can open the app, find their area and immediately see how many problems are open nearby, how their ward compares with the city, and what neighbours have reported. Reporting a problem takes a pin on the map, a category and a short description, and the report is stored in a Neon PostgreSQL database.

Data preparation remains central to the quality of the results. Separating Resolved from Rejected changed the city fix rate from the 78.9% reported by the first version to 68.0%, converting to IST corrected dates and hours, and joining the pothole layer on ID removed double counting.

Future work can refresh the data on a schedule to build a multi-year history, send resident reports to BMC through an official channel, add photos to reports, and move the browser-side aggregation to SQL views in Neon as the data grows.

6. References

[1] Brihanmumbai Municipal Corporation, “MARG_Application_Prod (MapServer): layers 0 COMPLAINTS_REGISTRATION, 16 POTHOLE_REGISTRATION and 2 Ward_Boundary,” MARG ArcGIS REST Services Directory, 2026. Accessed: Oct. 11, 2026. [Online]. Available: https://marg.mcgm.gov.in/server/rest/services/MARG_APP/MARG_Application_Prod/MapServer

[2] Brihanmumbai Municipal Corporation, “Pothole QuickFix dashboard,” BMC MARG. Accessed: Oct. 11, 2026. [Online]. Available: https://marg.mcgm.gov.in/BMC_C_Pothole_Dashboard/

[3] Praja Foundation, “Report on the status of civic services, environmental and climate issues in Mumbai 2026,” Praja Found., Mumbai, India, 2026. [Online]. Available: https://praja.org/praja_docs/praja_downloads/Report%20on%20the%20Status%20of%20Civic%20Services,%20Environmental%20and%20Climate%20Issues%20in%20Mumbai%202026.pdf